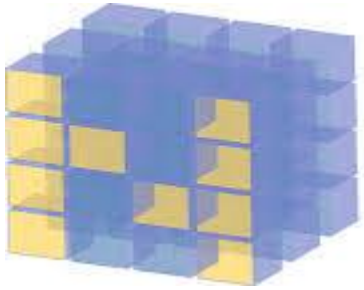
Matriz

Multidimensional

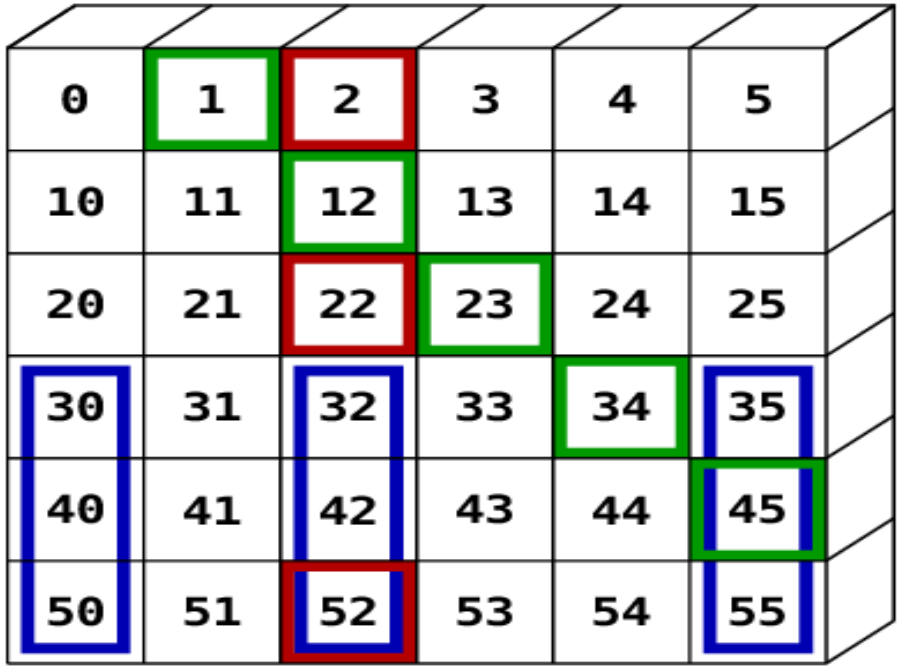


NumPy

```
>>> a[(0,1,2,3,4), (1,2,3,4,5)]
array([1, 12, 23, 34, 45])

>>> a[3:, [0,2,5]]
array([[30, 32, 35],
       [40, 42, 45],
       [50, 52, 55]])

>>> mask = np.array([1,0,1,0,0,1], dtype=bool)
>>> a[mask, 2]
array([2, 22, 52])
```



NumPy

Python es un gran lenguaje de programación de propósito general por sí mismo, pero con la ayuda de algunas bibliotecas populares (numpy, scipy, matplotlib) se convierte en un entorno poderoso para el cálculo científico.

Numpy es la biblioteca central para computación científica en Python. Proporciona un objeto de **matriz multidimensional de alto rendimiento** y herramientas para trabajar con estas matrices.



https://www.youtube.com/watch?v=Zhr0rWYuqfk

Redes neuronales convolucionales CS231n para reconocimiento visual

Python Numpy Tutorial

Este tutorial fue contribuido por Justin Johnson .

http://cs231n.github.io/python-numpy-tutorial/#numpy-arrays

Arrays Numpy

Una matriz numpy es una cuadrícula de valores del mismo tipo, y está indexada por una tupla de enteros no negativos.
El número de dimensiones es el *rango* de la matriz;
la *forma* (shape) una matriz es una tupla de enteros que da el tamaño de la matriz a lo largo de cada dimensión.

Podemos inicializar matrices numpy desde listas anidadas de Python y acceder a elementos mediante corchetes:

```
import numpy as np

a = np.array([1, 2, 3])   # Create a rank 1 array
print(type(a))            # Prints "<class 'numpy.ndarray'>"
print(a.shape)            # Prints "(3,)"
print(a[0], a[1], a[2])   # Prints "1 2 3"
a[0] = 5                  # Change an element of the array
print(a)                  # Prints "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]])    # Create a rank 2 array
print(b.shape)                     # Prints "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0])   # Prints "1 2 4"
```

Numpy también proporciona muchas funciones para crear matrices:

```
import numpy as np

a = np.zeros((2,2))   # Create an array of all zeros
print(a)              # Prints "[[ 0.  0.]
                      #          [ 0.  0.]]"

b = np.ones((1,2))    # Create an array of all ones
print(b)              # Prints "[[ 1.  1.]]"

c = np.full((2,2), 7)  # Create a constant array
print(c)               # Prints "[[ 7.  7.]
                       #          [ 7.  7.]]"

d = np.eye(2)         # Create a 2x2 identity matrix
print(d)              # Prints "[[ 1.  0.]
                      #          [ 0.  1.]]"

e = np.random.random((2,2))  # Create an array filled with random values
print(e)                     # Might print "[[ 0.91940167  0.08143941]
                             #               [ 0.68744134  0.87236687]]"
```

```python
import numpy as np

x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Elementwise sum; both produce the array
# [[ 6.0  8.0]
#  [10.0 12.0]]
print(x + y)
print(np.add(x, y))

# Elementwise difference; both produce the array
# [[-4.0 -4.0]
#  [-4.0 -4.0]]
print(x - y)
print(np.subtract(x, y))

# Elementwise product; both produce the array
# [[ 5.0 12.0]
#  [21.0 32.0]]
print(x * y)
print(np.multiply(x, y))
```

[1, 2] [5, 6]

[3, 4] [7, 8]

A diferencia de MATLAB, * es la multiplicación por elementos, no la multiplicación de matrices.

En su lugar, utilizamos la función **dot** para calcular los productos internos de los vectores, para multiplicar un vector por una matriz y para multiplicar las matrices.

dot está disponible como una función en el módulo numpy y como un método de instancia de objetos de matriz:

```
# Matrix / vector product; both produce the rank 1 array [29 67]
print(x.dot(v))
print(np.dot(x, v))

# Matrix / matrix product; both produce the rank 2 array
# [[19 22]
#  [43 50]]
print(x.dot(y))
print(np.dot(x, y))
```

Numpy proporciona muchas funciones útiles para realizar cálculos en arreglos;
Una de las más útiles es `sum`:

```
import numpy as np

x = np.array([[1,2],[3,4]])

print(np.sum(x))  # Compute sum of all elements; prints "10"
print(np.sum(x, axis=0))  # Compute sum of each column; prints "[4 6]"
print(np.sum(x, axis=1))  # Compute sum of each row; prints "[3 7]"
```

Práctica N01

- Puedes visitar este tutorial

https://pharos.sh/tutorial-de-numpy-una-guia-sencilla-basada-en-ejemplos/

- Experimentar los ejemplos de esta web

http://cs231n.github.io/python-numpy-tutorial/#numpy-arrays

- O visualizar el vídeo de introducción a NumPy

https://www.youtube.com/watch?v=8JfDAm9y_7s

Practiques d' Estadística bàsica

Amb w3schools

Temperatura	Colisiones 1	Colisiones 2	Colisiones 3	Promedio colisiones	Prom/Temp
5	47	51	54	50,67	10,1
10	111	105	109	108,33	10,8
20	208	217	224	216,33	10,8
30	318	339	305	320,67	10,7
40	428	433	402	421,00	10,5
50	555	523	518	532,00	10,6
60	565	621	633	606,33	10,1
70	719	716	690	708,33	10,1
80	832	797	802	810,33	10,1
90	945	902	887	911,33	10,1
100	969	996	1018	994,33	9,9

Media

(Promedio) Suma de datos dividido entre la cantidad de los mismos.

$$\bar{x} = \frac{\sum x}{N}$$

Moda

Dato que mas se repite. Si son dos es bimodal, si son 3 es trimodal.

Mediana

Dato central. Si son dos se saca la media de estos.

$$med_x$$

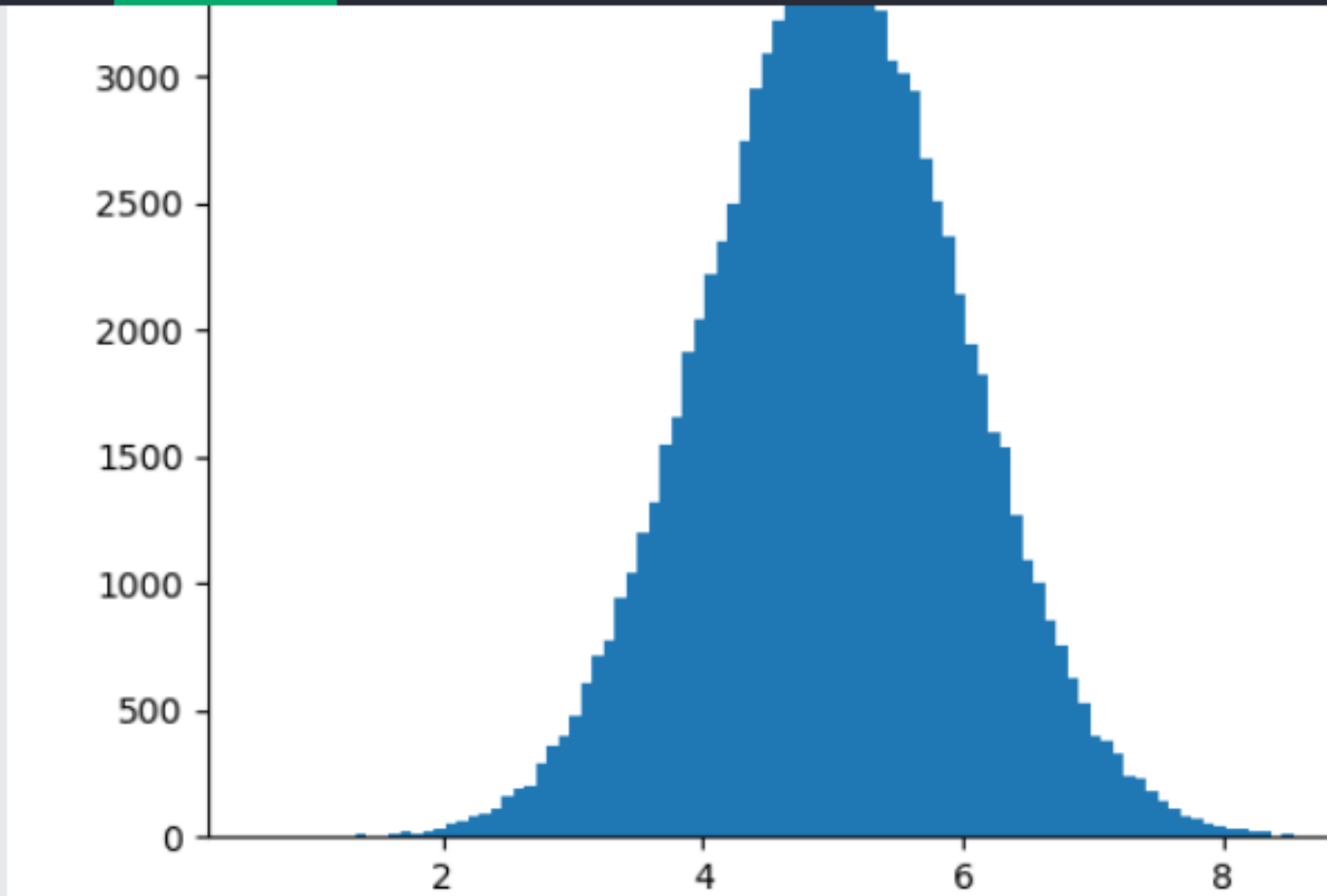
https://www.w3schools.com/python/python_ml_getting_started.asp

HTML CSS JAVASCRIPT SQL PYTHON JAVA PHP HOW TO W3.CSS C C++

Matplotlib Pie Charts

Machine Learning

Getting Started
Mean Median Mode
Standard Deviation
Percentile
Data Distribution
Normal Data Distribution
Scatter Plot
Linear Regression
Polynomial Regression
Multiple Regression
Scale



Per on començar?

En aquest tutorial tornarem a les matemàtiques i les estadístiques d'estudi, i a com calcular nombres importants a partir de conjunts de dades.

També aprendrem a utilitzar diversos mòduls de Python per obtenir les respostes que necessitem.

I aprendrem a fer funcions que siguin capaços de predir el resultat en funció del que hem après.

Conjunt de dades

En la ment d'un ordinador, un conjunt de dades és qualsevol col·lecció de dades. Pot ser qualsevol cosa, des d'una matriu fins a una base de dades completa.

Exemple de matriu:

```
[99,86,87,88,111,86,103,87,94,78,77,85,86]
```

Vector o matriu d'una dimensió

Matriu bidimensional

Podem endevinar que el valor mitjà de la velocitat està probablement al voltant de 80 o 90, i també podem determinar el valor més alt i el valor més baix, però què més podem fer?

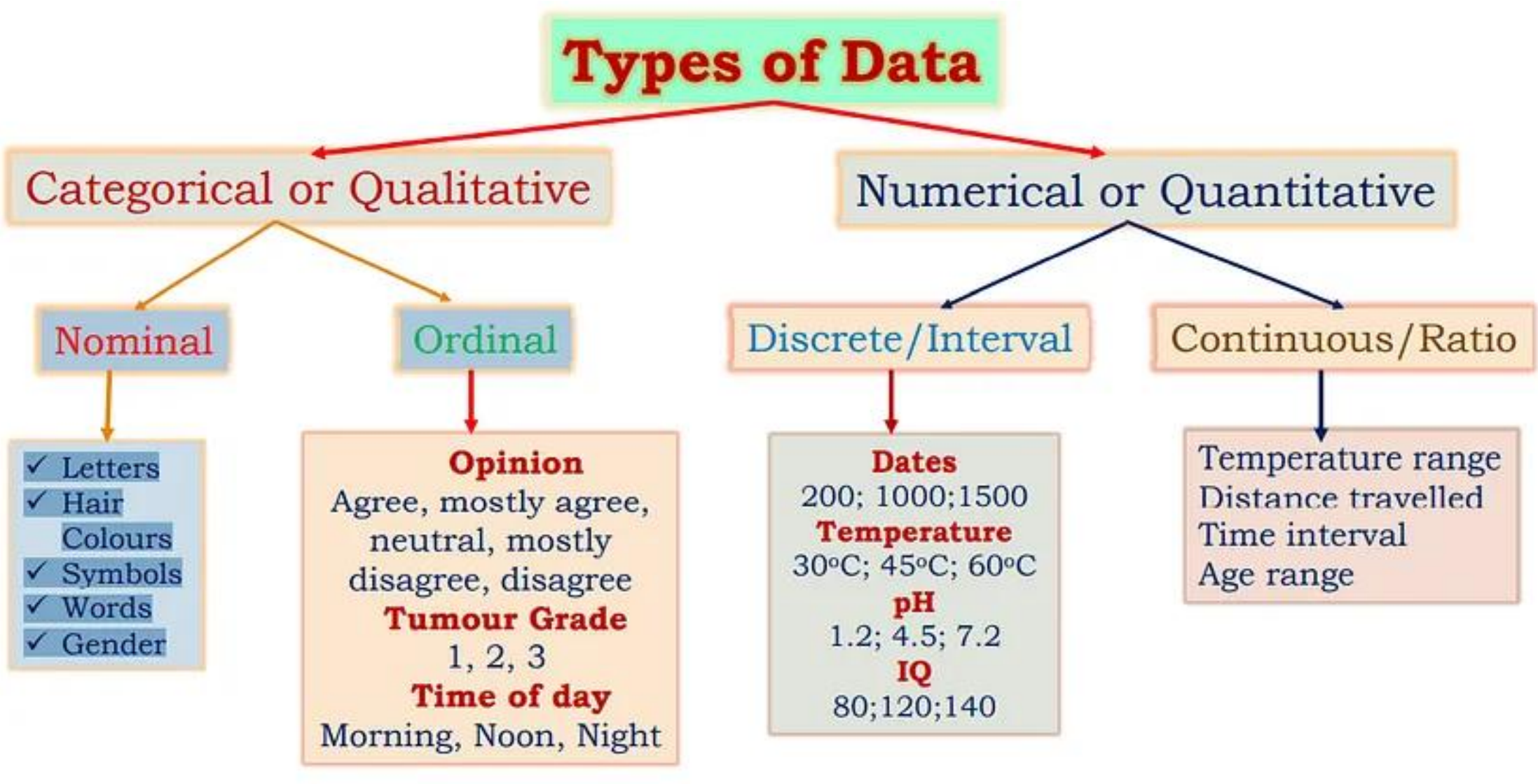
Carname	Color	Edat	Velocitat	AutoPass
BMW	vermell	5	99	Y
Volvo	negre	7	86	Y
VW	gris	8	87	N
VW	blanc	7	88	Y
Ford	blanc	2	111	Y
VW	blanc	17	86	Y
Tesla	vermell	2	103	Y
BMW	negre	9	87	Y
Volvo	gris	4	94	N
Ford	blanc	11	78	N
Toyota	gris	12	77	N
VW	blanc	9	85	N
Toyota	blau	6	86	Y

I mirant la taula podem veure que el **color més popular** és el blanc, i el cotxe més antic té 17 anys,
però què passaria si poguéssim **predir** si un cotxe tindrà un AutoPass, només mirant els altres valors? Per això serveix l'aprenentatge automàtic! Analitzant dades i predient el resultat!

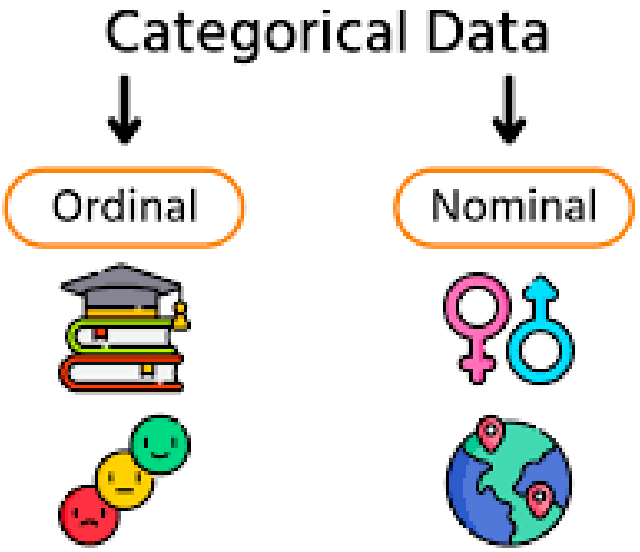
Carname	Color	Edat	Velocitat	AutoPass
BMW	vermell	5	99	Y
Volvo	negre	7	86	Y
VW	gris	8	87	N
VW	blanc	7	88	Y
Ford	blanc	2	111	Y
VW	blanc	17	86	Y
Tesla	vermell	2	103	Y

A l'aprenentatge automàtic és habitual treballar amb conjunts de dades molt grans. En aquest tutorial intentarem que sigui el més fàcil possible la comprensió dels diferents conceptes d'aprenentatge automàtic, i treballarem amb petits conjunts de dades fàcils d'entendre.

Tipus de Dades



Tipus de dades



Per analitzar les dades, és important saber amb quin tipus de dades estem tractant.

Podem dividir els tipus de dades en tres categories principals:

- **Numèric**
- **categòric**
- **Ordinal**

Les dades numèriques són números i es poden dividir en dues categories numèriques:

- Dades discretes
  : nombres limitats a nombres enters. Exemple: el nombre de cotxes que hi passen.
- Dades contínues
  : nombres que tenen un valor infinit. Exemple: el preu d'un article o la mida d'un article

Les dades categòriques són valors que no es poden comparar entre si. Exemple: un valor de color o qualsevol valor sí/no.

Les dades ordinals són com les dades categòriques, però es poden comparar les unes amb les altres. Exemple: notes escolars on A és millor que B i així successivament.

Si coneixeu el tipus de dades de la vostra font de dades, podreu saber quina tècnica utilitzareu per analitzar-les.

Mitjana, Mediana i Moda

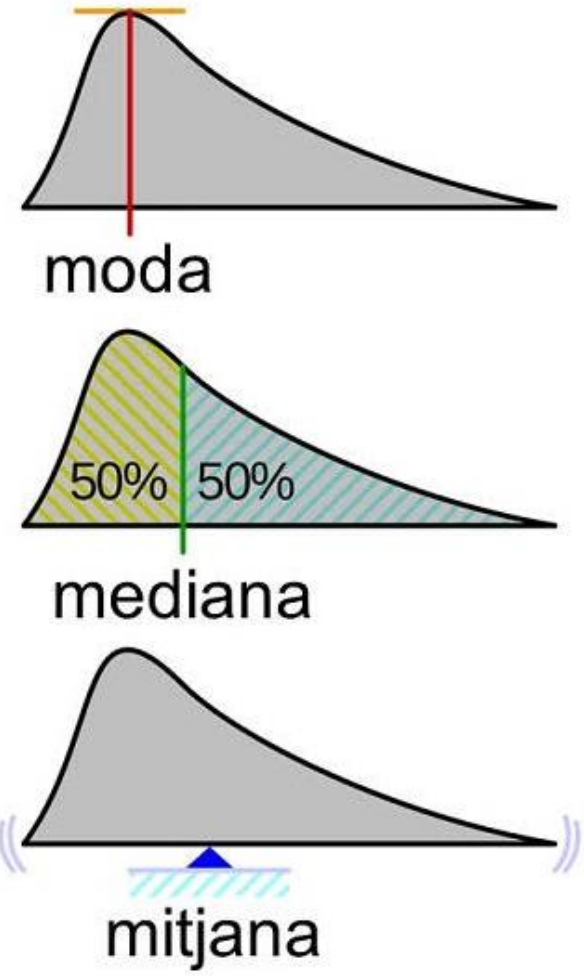
Què podem aprendre observant un grup de nombres?
En l'aprenentatge automàtic (i en les matemàtiques) sovint hi ha tres valors que ens interessen:

- **Mitjana** : el valor mitjà
- **Mediana** : el valor del punt mitjà
- **Mode** : el valor més comú

Exemple: Hem registrat la velocitat de 13 cotxes:

```
speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]
```

Quin és el valor de velocitat mitjà, mitjà o més comú?



Mitjana

Per calcular la mitjana, troba la suma de tots els valors i divideix la suma pel nombre de valors:

(99+86+87+88+111+86+103+87+94+78+77+85+86) / 13 = 89.77

El mòdul NumPy té un mètode per a això. Obteniu informació sobre el mòdul NumPy al nostre Tutorial NumPy .

Utilitzeu el `mean()`mètode NumPy per trobar la velocitat mitjana:

```
import numpy

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = numpy.mean(speed)

print(x)
```

Mediana

El valor mitjà és el valor del mig, després d'haver ordenat tots els valors:
77, 78, 85, 86, 86, 86, 87, 87, 88, 94, 99, 103, 111

És important que els números estiguin ordenats abans de trobar la mediana.
El mòdul NumPy té un mètode per a això:

Exemple
Utilitzeu el mètode NumPy `median()` per trobar el valor mitjà:

```
import numpy
speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = numpy.median(speed)

print(x)
```

Prova-ho tu mateix »

Si hi ha dos nombres al mig, divideix la suma d'aquests nombres per dos.

```
77, 78, 85, 86, 86, 86, 87, 87, 94, 98, 99, 103
```

```
(86 + 87) / 2 = 86.5
```

Exemple

Utilitzant el mòdul NumPy:

```
import numpy

speed = [99,86,87,88,86,103,87,94,78,77,85,86]

x = numpy.median(speed)

print(x)
```

Moda

El valor Moda és el valor que apareix més vegades:

99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86 = 86

El mòdul SciPy té un mètode per a això. Obteniu informació sobre el mòdul SciPy al nostre tutorial SciPy .

Exemple

Utilitzeu el mètode SciPy `mode()` per trobar el número que apareix més:

```
from scipy import stats

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = stats.mode(speed)

print(x)
```

Prova-ho tu mateix »

Què és la desviació estàndard o típica?

La desviació estàndard és un nombre que descriu la variabilitat dels valors respecte la mitjana.

Una desviació estàndard baixa significa que la majoria dels nombres estan a prop del valor mitjà (mitjana).
Una desviació estàndard alta significa que els valors es distribueixen en un rang més ampli.

Exemple: Aquesta vegada hem registrat la velocitat de 7 cotxes:

```
speed = [86,87,88,86,87,85,86]
```

La desviació estàndard és: 0.9

Això vol dir que la majoria dels valors es troben dins del rang de 0,9 del valor mitjà, que és 86,4.
Fem el mateix amb una selecció de números amb un rang més ampli:

```
speed = [32,111,138,28,59,77,97]
```

La desviació estàndard és: 37.85

Desviació típica

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Això vol dir que la majoria dels valors es troben dins del rang de 37,85 del valor mitjà, que és 77,4.

Com podeu veure, una desviació estàndard més alta indica que els valors estan repartits en un rang més ampli.

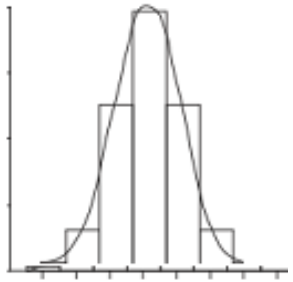
El mòdul NumPy té un mètode per calcular la desviació estàndard:

```
import numpy

speed = [86,87,88,86,87,85,86]

x = numpy.std(speed)

print(x)
```

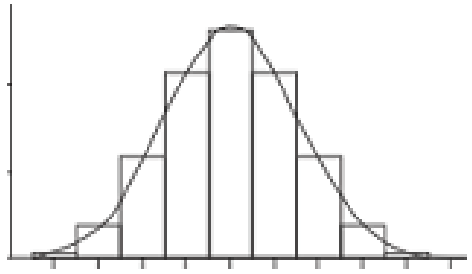


```
import numpy

speed = [32,111,138,28,59,77,97]

x = numpy.std(speed)

print(x)
```



Desviació estàndar

Com hem après, la fórmula per trobar la desviació estàndard és l'arrel quadrada de la variància:
√1432.25 = 37.85

O, com a l'exemple anterior, utilitzeu NumPy per calcular la desviació estàndard:

Exemple

Utilitzeu el mètode NumPy `std()`per trobar la desviació estàndard:

```
import numpy

speed = [32,111,138,28,59,77,97]

x = numpy.std(speed)

print(x)
```

Variància

La variància és un altre nombre que indica la distribució dels valors.

De fet, si agafeu l'arrel quadrada de la variància, obteniu la desviació estàndard!
O al revés, si multipliqueu la desviació estàndard per si mateixa, obteniu la variància!
Per calcular la variància cal fer el següent:

1. Trobeu la mitjana:

```
(32+111+138+28+59+77+97) / 7 = 77.4
```

2. Per a cada valor: troba la diferència amb la mitjana:

```
 32 - 77.4 = -45.4
111 - 77.4 =  33.6
138 - 77.4 =  60.6
 28 - 77.4 = -49.4
 59 - 77.4 = -18.4
 77 - 77.4 = - 0.4
 97 - 77.4 =  19.6
```

3. Per a cada diferència: troba el valor quadrat:

```
(-45.4)² = 2061.16
 (33.6)² = 1128.96
 (60.6)² = 3672.36
(-49.4)² = 2440.36
(-18.4)² =  338.56
(- 0.4)² =    0.16
 (19.6)² =  384.16
```

4. La variància és el nombre mitjà d'aquestes diferències al quadrat:

```
(2061.16+1128.96+3672.36+2440.36+338.56+0.16+384.16) / 7 = 1432.20
```

Exemple

Utilitzeu el mètode NumPy `var()`per trobar la variància:

```
import numpy

speed = [32,111,138,28,59,77,97]

x = numpy.var(speed)

print(x)
```

Variança | $\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$

Desviació típica | $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$

La desviació estàndard sovint es representa amb el símbol Sigma: σ
La variància sovint es representa amb el símbol Sigma quadrat: σ^2

Percentils o Quartils

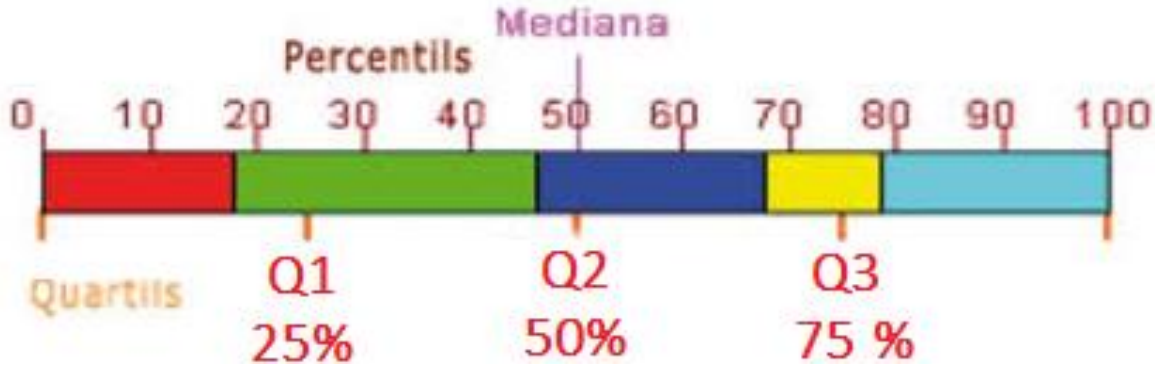
És una mesura de posició no central que permet conèixer altres punts característics de la distribució que no són els valors centrals. El quantil d'ordre p d'una distribució (amb 0 < p < 1) és el valor de la variable xp que marca un tall de manera que una proporció p de valors de la població és menor o igual que xp.

Exemple: suposem que tenim una sèrie d'edats de totes les persones que viuen al carrer.

ages=[5,31,43,48,50,41,7,11,15,39,80,82,32,2,8,6,25,36,27,61,31]

Quin és el percentil 75.? La resposta és 43, és a dir, el 75% de la gent té 43 anys o menys.

El mòdul NumPy té un mètode per trobar el percentil especificat:

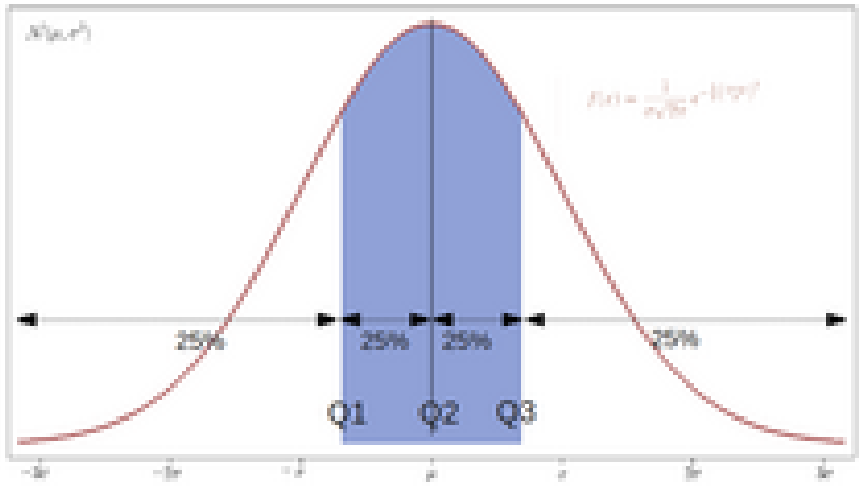


Exemple

Utilitzeu el mètode NumPy `percentile()`per trobar els percentils:

```
import numpy
ages=[5,31,43,48,50,41,7,11,15,39,80,82,32,2,8,6,25,36,27,61,31]
x=numpy.percentile(ages,75)
print(x)
```

Els quantils 0.25, 0.50 i 0.75 de la distribució normal. Més coneguts com els quartils Q_1, Q_2 i Q_3, divideixen la distribució en quatre blocs, cadascun dels quals conté el 25% de les dades.



Exemple

Quina edat limita el 90% de la gent més jove?

```
import numpy
ages=[5,31,43,48,50,41,7,11,15,39,80,82,32,2,8,6,25,36,27,61,31]
x=numpy.percentile(ages, 90)
print(x)
```

```
61.0
[2, 5, 6, 7, 8, 11, 15, 25, 27, 31, 31, 32, 36, 39, 41, 43, 48, 50, 61, 80, 82]
```

Distribució de dades

A l'inici d'aquest tutorial hem treballat amb quantitats molt petites de dades en els nostres exemples, només per entendre els diferents conceptes

.

Al món real, els conjunts de dades són molt més grans, però pot ser difícil reunir dades del món real, almenys en una fase inicial d'un projecte.

Com podem obtenir conjunts de dades grans?

Per crear grans conjunts de dades per fer proves, utilitzem el mòdul Python NumPy, que inclou una sèrie de mètodes per crear conjunts de dades aleatoris, de qualsevol mida.

Exemple

Creeu una matriu que contingui 250 flotants aleatoris entre 0 i 5:

```
import numpy

x = numpy.random.uniform(0.0, 5.0, 250)

print(x)
```

Què és un Histograma ?

Un histograma és una **representació gràfica** associada a una variable estadística contínua (o discreta amb molts valors diferents) per visualitzar les **freqüències** de les dades.

Es construeix a partir d'una taula de freqüències o distribució de freqüències amb les dades agrupades en classes.

$7,4$	$5,2$	4	$8,1$	$3,7$
$6,8$	10	$3,2$	6	$5,5$
$6,2$	$7,8$	$6,4$	4	1
5	$5,9$	$5,5$	7	5

La taula de freqüències d'aquestes dades és

Interval	Freqüència absoluta
$[0,2]$	1
$(2,4]$	4
$(4,6]$	7
$(6,8]$	6
$(8,10]$	2
Total	**20**

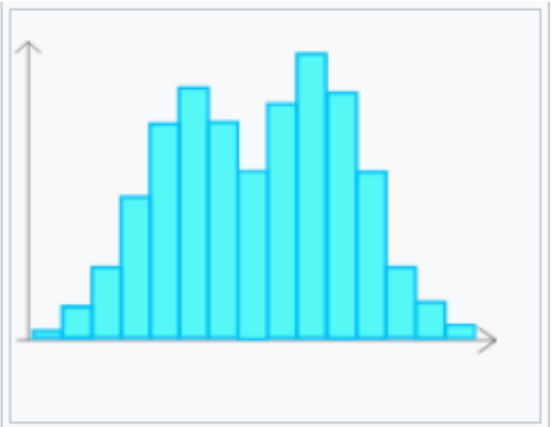


Figura 1. Exemple d'un histograma construït a partir de 100 dades, on pot veure's la distribució de freqüències de la variable estudiada.

Histograma

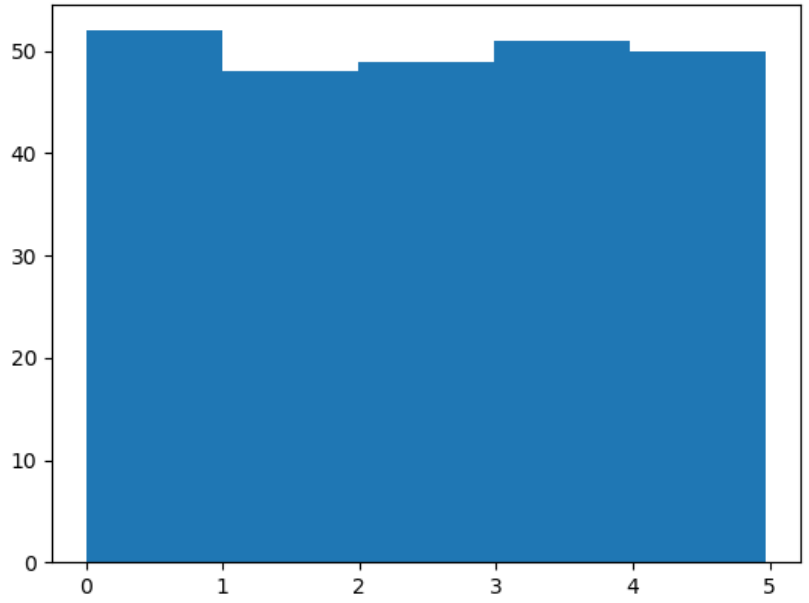
Per visualitzar el conjunt de dades podem dibuixar un histograma amb les dades que hem recollit.
Utilitzarem el mòdul de Python Matplotlib per dibuixar un histograma.
Obteniu informació sobre el mòdul Matplotlib al nostre Tutorial Matplotlib .

Exemple
Dibuixa un histograma:

```
import numpy
import matplotlib.pyplot as plt

x = numpy.random.uniform(0.0, 5.0, 250)

plt.hist(x, 5)
plt.show()
```



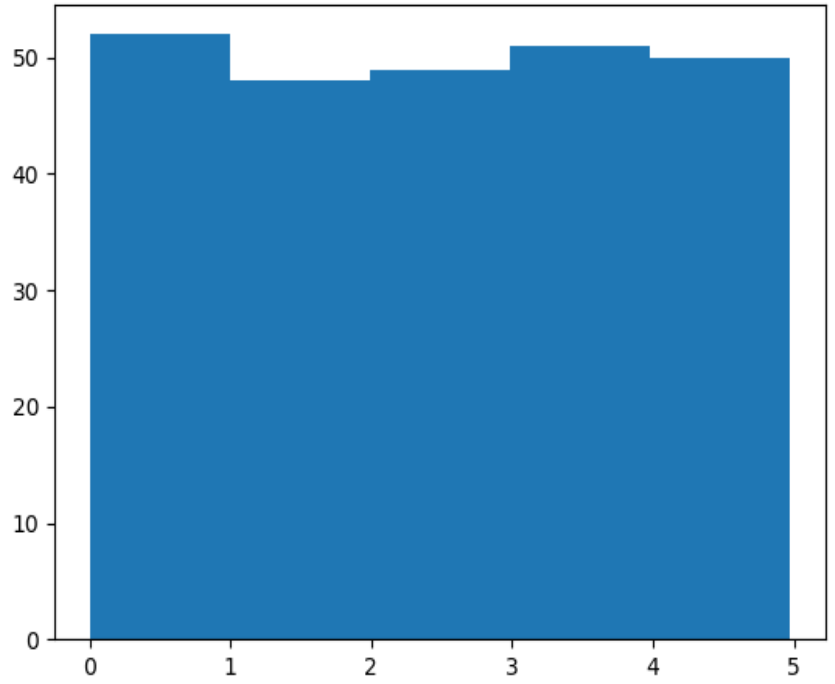
Histograma explicat

Utilitzem la matriu de l'exemple anterior per dibuixar un histograma amb 5 barres.
La primera barra representa quants valors de la matriu estan entre 0 i 1.
La segona barra representa quants valors estan entre 1 i 2.
Etc.

El que ens dóna aquest resultat:

- 52 valors estan entre 0 i 1
- 48 valors estan entre 1 i 2
- 49 valors estan entre 2 i 3
- 51 valors estan entre 3 i 4
- 50 valors estan entre 4 i 5

Nota: els valors de la matriu són números aleatoris i no mostraran exactament el mateix resultat a l'ordinador.



Distribucions de Big Data

Una matriu que conté 250 valors no es considera molt gran, però ara ja sabeu com crear un conjunt aleatori de valors i, canviant els paràmetres, podeu crear el conjunt de dades tan gran com vulgueu.

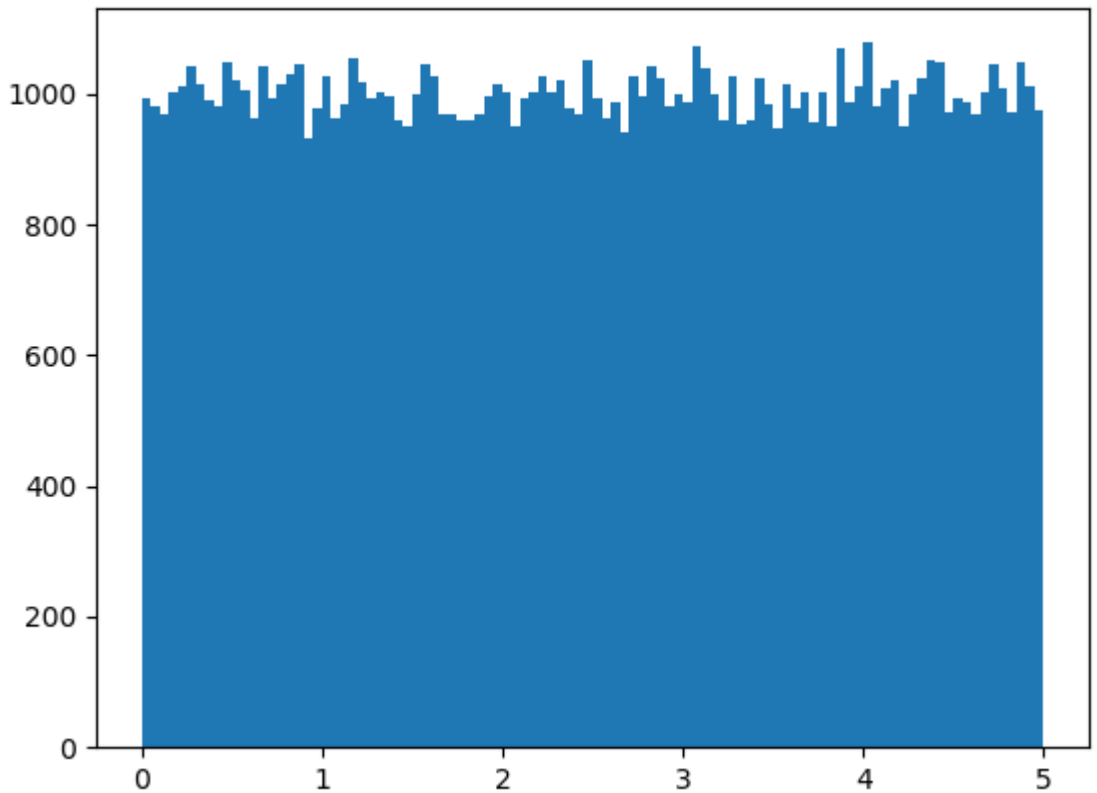
Exemple

Creeu una matriu amb 100.000 números aleatoris i visualitzeu-los mitjançant un histograma amb 100 barres:

```
import numpy
import matplotlib.pyplot as plt

x = numpy.random.uniform(0.0, 5.0, 100000)

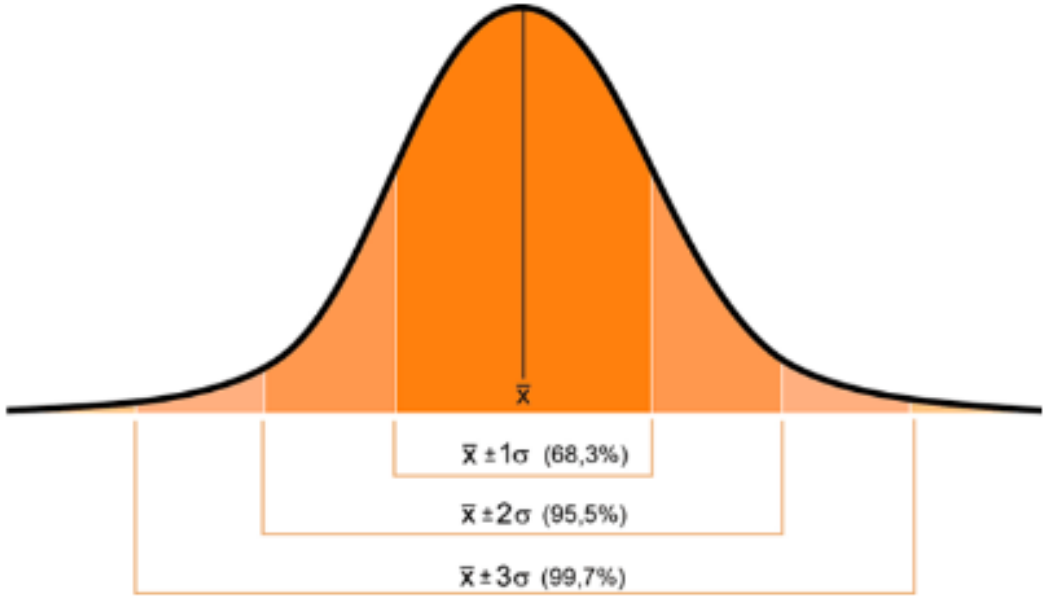
plt.hist(x, 100)
plt.show()
```

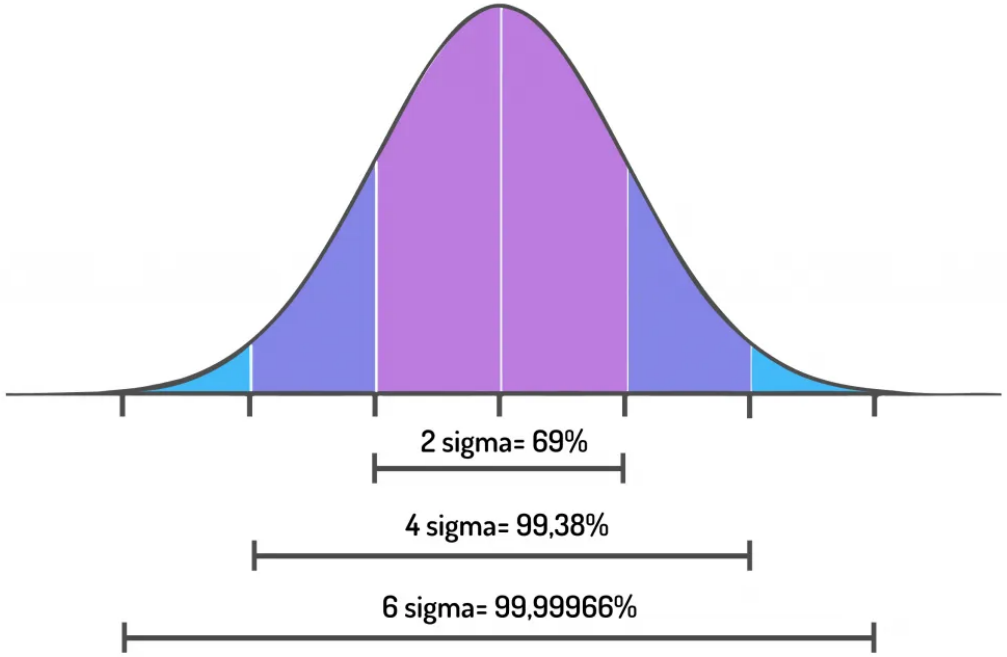

1000
800
600
400
200
0
0
1
2
3
4
5

Què és una distribució normal ?

La distribució normal, també coneguda com a Campana de Gauss, és una distribució freqüent en probabilitat i estadística.

Compleix que prop del 60% de les dades es concentren al voltant de la mitjana, amb valors menys probables als extrems. Aquesta distribució es visualitza amb una gràfica on el punt més alt és al centre i els extrems tenen valors més grans o petits que la mitjana.





Histograma d'una distribució Normal

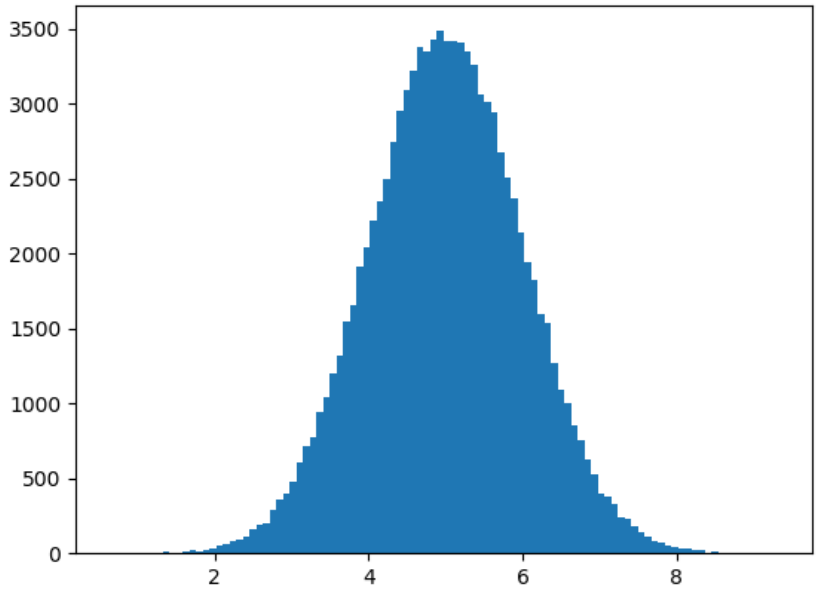
En el apartat anterior vam aprendre a crear una matriu completament aleatòria, d'una mida determinada i entre dos valors donats.
En aquest apartat aprendrem a crear una matriu on els valors es concentren al voltant d'un valor determinat.
En teoria de la probabilitat, aquest tipus de distribució de dades es coneix com a *distribució de dades normal* , o *distribució de dades gaussiana* , després del matemàtic Carl Friedrich Gauss que va inventar la fórmula d'aquesta distribució de dades.

Exemple
Una distribució de dades normal típica:

```
import numpy
import matplotlib.pyplot as plt

x = numpy.random.normal(5.0, 1.0, 100000)

plt.hist(x, 100)
plt.show()
```



Nota: un gràfic de distribució normal també es coneix com a *corba de campana* a causa de la seva forma característica d'una campana.

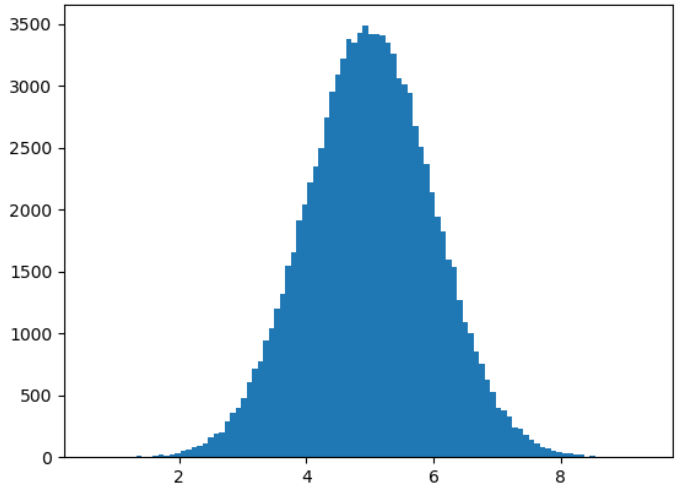
Funció Histograma explicada

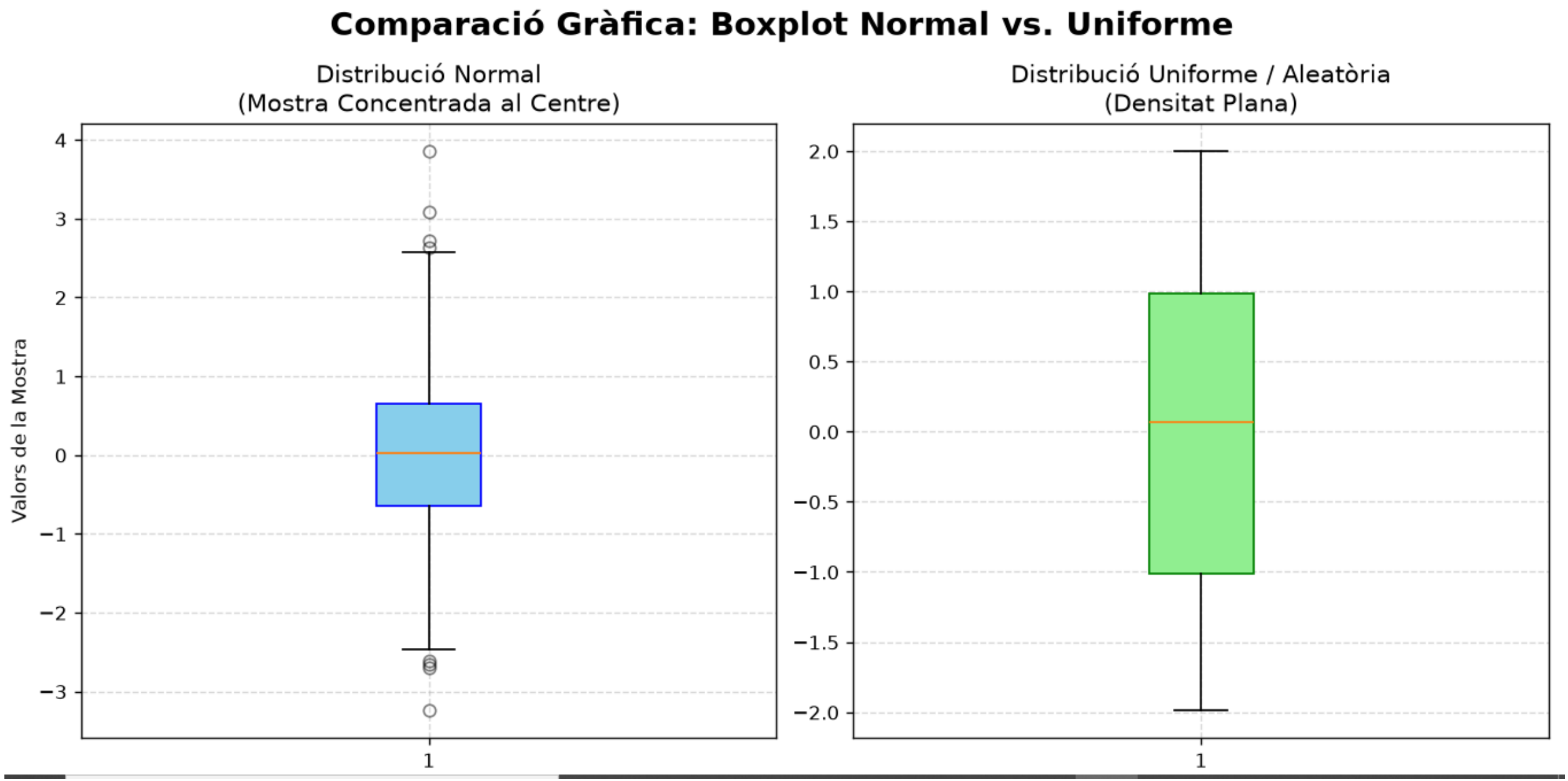
- Utilitzem la matriu del `numpy.random.normal()` mètode, amb 100.000 valors, per dibuixar un histograma amb 100 barres.
- Especifiquem que el valor mitjà és 5,0 i la desviació estàndard és 1,0.

```
x = numpy.random.normal(5.0, 1.0, 100000)
```

- Això vol dir que els valors s'han de concentrar al voltant de 5,0 i rarament més lluny d'1,0 de la mitjana.

- I com podeu veure a l'histograma, la majoria de valors
- estan entre 4,0 i 6,0, amb una part superior
- aproximadament 5,0.




Comparació Gràfica: Boxplot Normal vs. Uniforme
Distribució Normal
(Mostra Concentrada al Centre)
Distribució Uniforme / Aleatòria
(Densitat Plana)
Valors de la Mostra
4
3
2
1
0
−1
−2
−3
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
1
1

Característica	**Mostra Aleatòria (Uniforme)**	**Distribució Normal**
Densitat central	Plana / Homogènia	Concentrada al voltant de la mediana
Presència d'Outliers	Inusual o inexistent	Esperada (aprox. 1% a 2% del total)
Bigotis (*Whiskers*)	Arriben gairebé sempre fins als límits extrems	S'aturen a 1,5 * IQR deixant l'espai per als outliers
Simetria visual	Simètrica (en mostres grans)	Molt simètrica per definició

1. Definició de les Mides Centrals

Abans de calcular els límits, es determinen els tres quartils d'un conjunt de dades ordenat:

1. **Primer Quartil (Q_1):** Valor per sota del qual cau el 25% de les dades (percentil 25).
2. **Tercer Quartil (Q_3):** Valor per sota del qual cau el 75% de les dades (percentil 75).

A partir d'aquests, es calcula el **Rang Interquartílic (IQR)**, que mesura l'dispersió del 50% central de les dades:

$$IQR = Q_3 - Q_1$$

2. Càlcul dels Límits Teòrics (*Fences*)

Per decidir quins punts són considerats normals i quins són atípics, es defineixen dues "tanques" o límits teòrics:

- **Límit Inferior Teòric (LI):**

$$LI = Q_1 - (1,5 \times IQR)$$

- **Límit Superior Teòric (LS):**

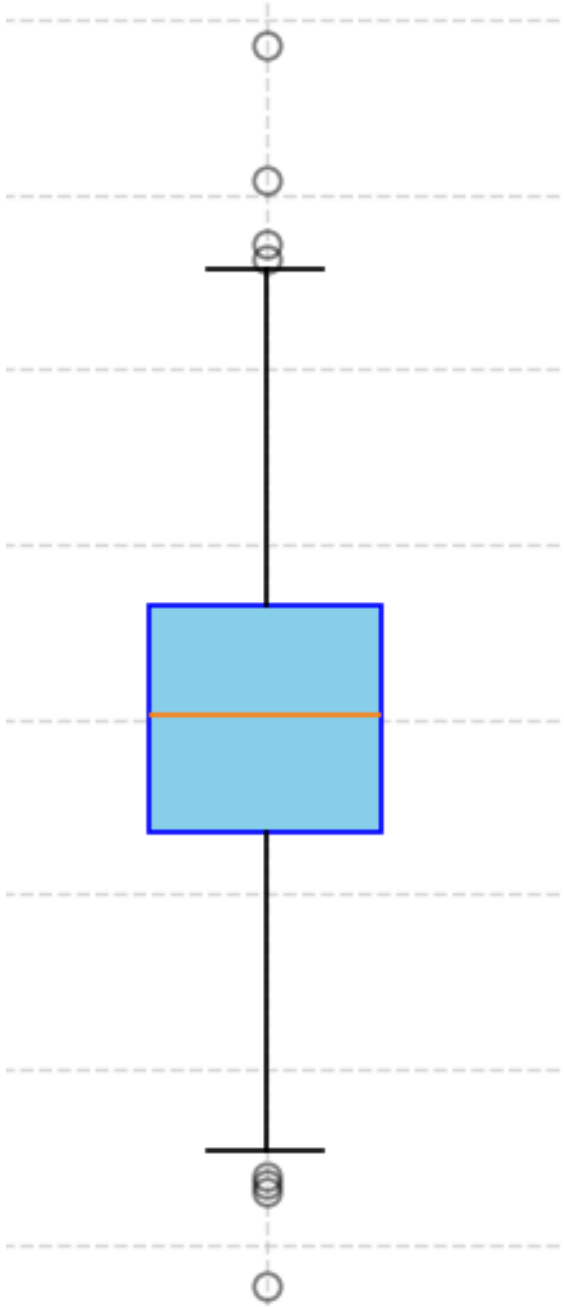
$$LS = Q_3 + (1,5 \times IQR)$$

3. Delimitació dels Bigotis i Definició d'Outliers

Els bigotis del *boxplot* **no s'estenen necessàriament fins a** LI **i** LS, sinó fins als valors reals de les dades més extrems que no superen aquestes tanques:

- **Extrem del Bigot Inferior:** El valor real de la mostra més petit que sigui mes gran o igual a LI.
- **Extrem del Bigot Superior:** El valor real de la mostra més gran que sigui mes petit o igual a LS.
- **Valors Atípics (*Outliers*):** Qualsevol dada x de la mostra que quedi estrictament fora dels límits teòrics:

$$x < LI \quad \text{o} \quad x > LS$$

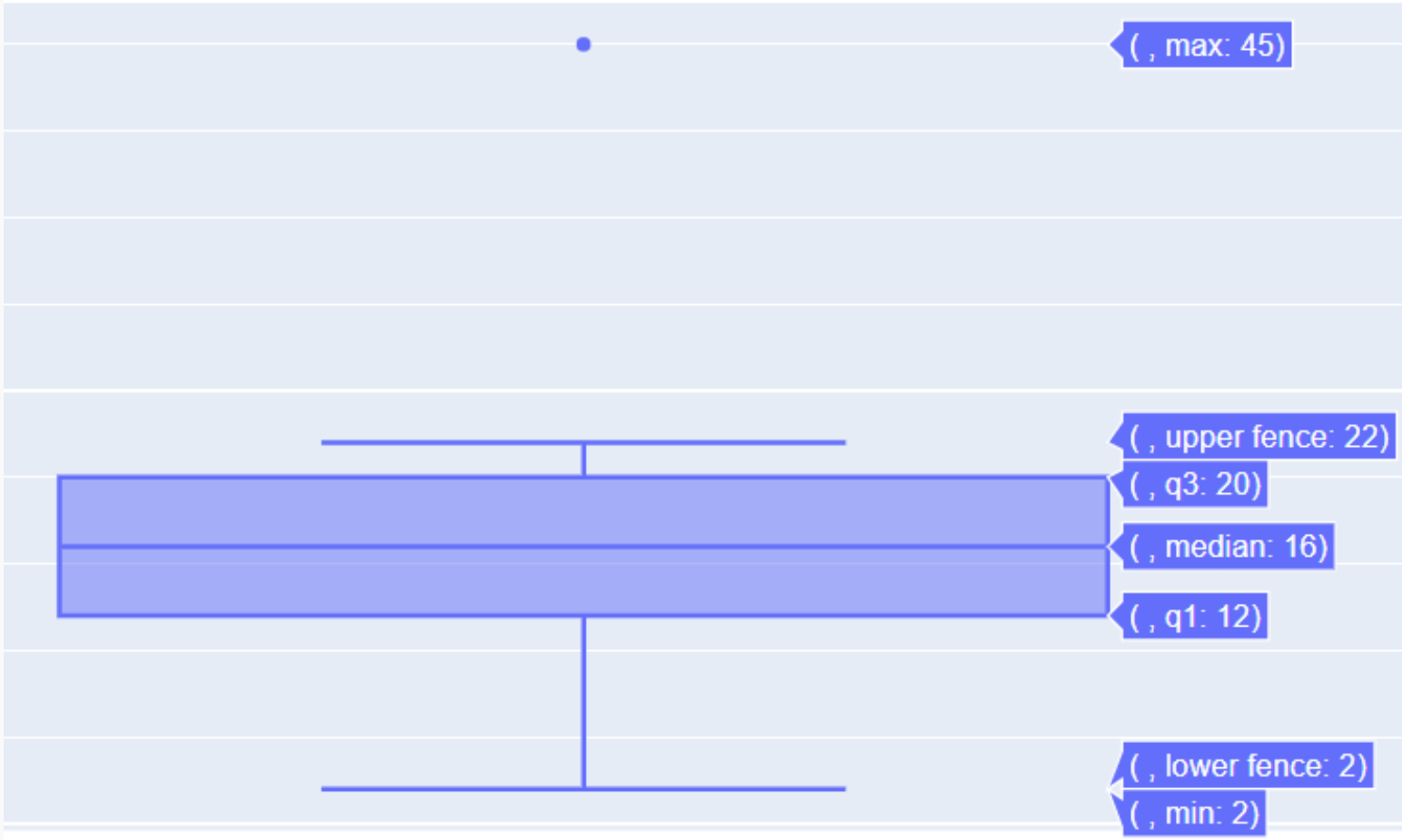


4. Exemple Numèric Pas a Pas

Considerem la seqüència ordenada de 10 dades següent:

$$\mathbf{2, 10, 12, 14, 15, 17, 19, 20, 22, 45}$$

1. **Trobar Q_1 i Q_3:**
   - $Q_1 = 12$
   - $Q_3 = 20$
2. **Calcular l'IQR:**
   - $IQR = 20 - 12 = 8$
3. **Calcular els límits teòrics ($1,5 \times IQR = 12$):**
   - $LI = 12 - 12 = 0$
   - $LS = 20 + 12 = 32$
4. **Determinar els bigotis i outliers:**
   - El valor més petit ≥ 0 és **2** (extrem del bigot inferior).
   - El valor més gran ≤ 32 és **22** (extrem del bigot superior).
   - El valor **45** és estrictament major que 32 ($45 > LS$), per la qual cosa es dibuixa com un **outlier**.



Plotly

```
mport pandas as pd
import plotly.express as px

df = pd.DataFrame({'A': [2, 10, 12, 14, 15, 17, 19, 20, 22, 45]})
fig = px.box(df, y="A")
fig.show()
```

MatplotLib

```
import pandas as pd
import matplotlib.pyplot as plt

bplot = plt.boxplot([2, 10, 12, 14, 15, 17, 19, 20, 22, 45])
plt.show()
```

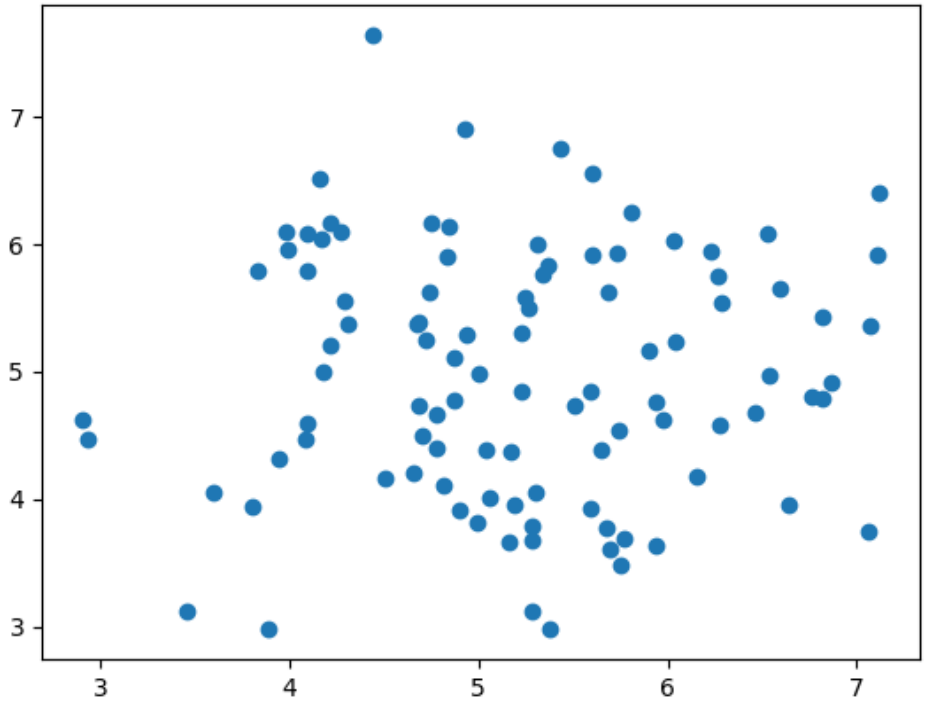
Gràfic de dispersió o de punts

Un diagrama de dispersió és un diagrama on cada valor del conjunt de dades es representa amb un punt.

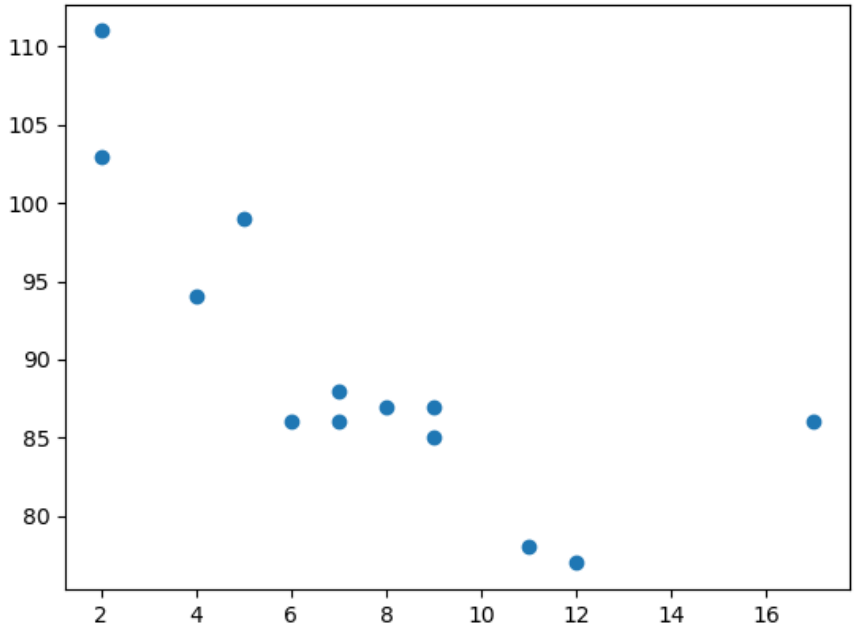
El mòdul Matplotlib té un mètode per dibuixar gràfics de dispersió, necessita dues matrius de la mateixa longitud, una per als valors de l'eix X i una altra per als valors de l'eix Y:

```
x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]
```

La matriu x representa l'edat de cada cotxe.
La matriu y representa la velocitat de cada cotxe.



Gràfic de dispersió



Utilitzeu el mètode `scatter()` per dibuixar un diagrama de dispersió:

```
import matplotlib.pyplot as plt

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

plt.scatter(x, y)
plt.show()
```

S'explica el diagrama de dispersió

L'eix X representa les edats i l'eix Y representa les velocitats.

El que podem llegir del diagrama és que els dos cotxes més ràpids tenien 2 anys i el cotxe més lent tenia 12 anys.

Nota: Sembla que com més nou és el cotxe, més ràpid es condueix, però això podria ser una coincidència, al cap i a la fi només hem matriculat 13 cotxes.

Distribucions aleatòries de dades

A l'aprenentatge automàtic, els conjunts de dades poden contenir milers, o fins i tot milions, de valors.
És possible que no tingueu dades del món real quan esteu provant un algorisme, potser haureu d'utilitzar valors generats aleatòriament.
Com hem après al capítol anterior, el mòdul NumPy ens pot ajudar amb això!
Creem dues matrius que s'omplen amb 1000 números aleatoris d'una distribució de dades normal.

La primera matriu tindrà la mitjana establerta a 5,0 amb una desviació estàndard d'1,0.

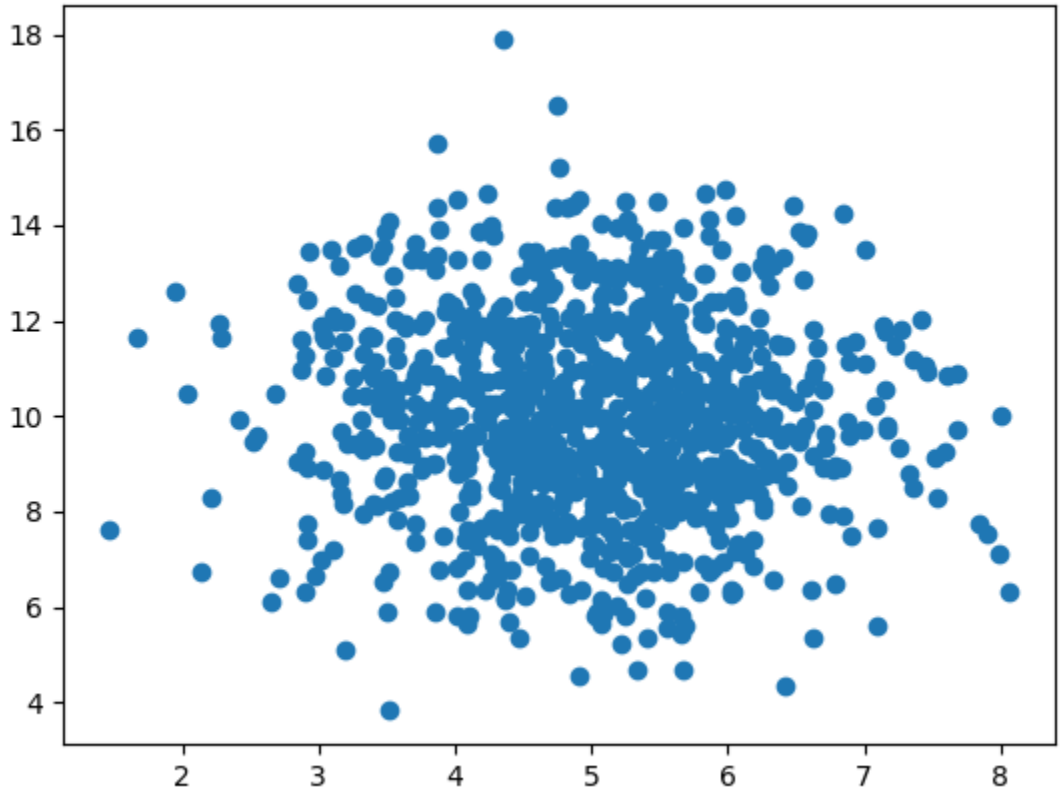
La segona matriu tindrà la mitjana establerta en 10,0 amb una desviació estàndard de 2,0:

Exemple
Un diagrama de dispersió amb 1000 punts:

```
import numpy
import matplotlib.pyplot as plt

x = numpy.random.normal(5.0, 1.0, 1000)
y = numpy.random.normal(10.0, 2.0, 1000)

plt.scatter(x, y)
plt.show()
```



S'explica el diagrama de dispersió

Podem veure que els punts es concentren al voltant del valor 5 a l'eix x, i 10 a l'eix y. També podem veure que la dispersió és més àmplia a l'eix y que a l'eix x.